

Chapter 9 – Recursive Functions

Slide 1 — What Is a Recursive Function?

Definition

A **recursive function** is a function that **calls itself** to solve a problem.

Instead of solving the entire problem at once, the function breaks it into **smaller versions of the same problem**.

A recursive function has two important parts:

1. **Base Case** — Stops the recursion.
2. **Recursive Case** — Calls the function again with a smaller or simpler value.

Analogy

Think of **Russian dolls**:

- You open one doll.
- Inside is another smaller doll.
- You continue opening them.
- Eventually, you reach the smallest doll.
- Then you stop.

The smallest doll is like the **base case**.

Small Example

None

```
void count(int n) {  
    if (n == 0)  
        return;  
  
    printf("%d ", n);  
    count(n - 1);  
}
```

Here, `count()` calls itself.

Remember

Recursion = A function solving a problem by calling itself.

Slide 2 — Why Are Recursive Functions Used?

Why use recursion?

Recursion is useful when a problem naturally consists of **smaller versions of itself**.

It can help with:

- Repeating a task with changing values.
- Breaking a problem into smaller parts.
- Mathematical problems such as factorial.
- Working with hierarchical structures.
- Problems where the same logic is applied repeatedly.

Important Statement

Recursion is useful when the solution to a problem can be expressed using a **smaller version of the same problem**.

Basic → Professional

Basic:

```
None  
count(n - 1);
```

The function calls itself with a smaller value.

Professional idea:

Recursive thinking is useful in problems where a large problem can naturally be divided into **smaller similar problems**.

Important

A recursive function must have a **stopping condition**.

Without one, the function can continue calling itself indefinitely.

Slide 3 — Base Case and Recursive Case

The Two Parts of Recursion

Every useful recursive function needs to answer two questions:

1. When should I stop?

This is the **base case**.

```
None
if (n == 0)
    return;
```

It prevents further function calls.

2. How should I continue?

This is the **recursive case**.

```
None
count(n - 1);
```

The function calls itself with a smaller value.

Example

```
None
void count(int n) {
    if (n == 0)
        return;

    printf("%d ", n);
    count(n - 1);
}
```

For:

None

`count(3)`

The calls become:

None

`count(3)`

↓

`count(2)`

↓

`count(1)`

↓

`count(0)`

↓

`STOP`

Important Statement

Base case stops recursion; recursive case continues recursion.

Memory Trick

Base → STOP

Recursive → AGAIN

Slide 4 — Recursion Flowchart

How Recursive Execution Works

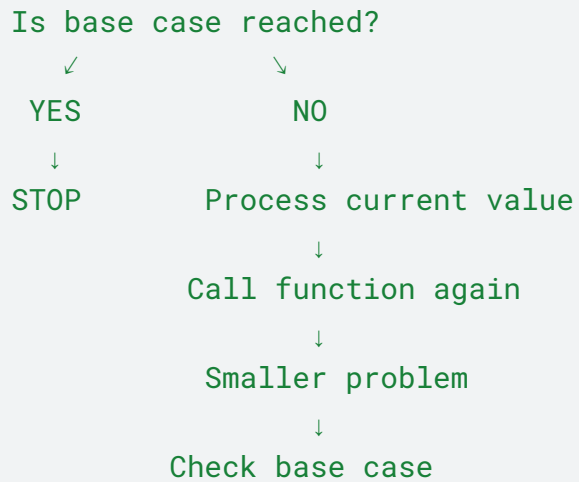
None

`START`

↓

`Function receives n`

↓



Example

None

```
#include <stdio.h>

void count(int n) {
    if (n == 0)
        return;

    printf("%d ", n);
    count(n - 1);
}

int main() {
    count(3);
    return 0;
}
```

Output

None

```
3 2 1
```

What Happens?

1. `count(3)` starts.
 2. `3` is printed.
 3. It calls `count(2)`.
 4. `2` is printed.
 5. It calls `count(1)`.
 6. `1` is printed.
 7. `count(0)` reaches the base case.
 8. Recursion stops.
-

Slide 5 — Witty Recap

1. Recursive Function

A function that calls itself during its execution.

2. Base Case

The condition that tells the function when to stop. Without a base case, recursion may continue indefinitely.

3. Recursive Case

The part where the function calls itself again with a smaller or simpler problem.

4. Progress Toward the Base Case

Every recursive call should move closer to the stopping condition.

5. Function Execution

The calls continue until the base case is reached, and then the recursion stops.

Questions

1. Theory MCQ — Current Topic

What is the main purpose of a base case in a recursive function?

- A. To start the program
- B. To create a variable

- C. To stop the recursion
- D. To call another function

💡 Hint: Every recursion needs a condition that tells it when to stop.

📖 Glossary:

Base Case — The condition that stops a recursive function from calling itself further.

Answer: C. To stop the recursion

2. Output Guess — Current Topic

None

```
#include <stdio.h>
```

```
void count(int n) {
```

```
    if (n == 0)
```

```
        return;
```

```
    printf("%d ", n);
```

```
    count(n - 1);
```

```
}
```

```
int main() {
```

```
count(3);  
  
return 0;  
  
}
```

What will be the output?

 Hint: Each call decreases **n** by **1**.

 Glossary:

Recursive Case — The part of a function where it calls itself again.

Answer:

None

3 2 1

3. Fill the Missing Code — Current Topic

None

```
#include <stdio.h>  
  
void count(int n) {  
    if (_____)  
        return;
```

```
printf("%d ", n);  
  
count(n - 1);  
  
}
```

Fill in the missing condition.

 Hint: The recursion should stop when **n** reaches zero.

 Glossary:

Condition — An expression that determines whether a particular action should happen.

Answer: **n == 0**

4. Jumbled Sentence — Previous Topic: Local and Global Variables

Arrange the words correctly:

inside / declared / A / local / function / is / variable / a

 Hint: Start with A.

 Glossary:

Local Variable — A variable declared inside a function or block.

Answer: A local variable is declared inside a function.

5. Fill in the Blank — Second Previous Topic: Multiple Parameters and Arguments

The actual values passed to a function during a function call are called _____.

 **Hint: They provide values to the parameters.**

 **Glossary:**

Arguments — The actual values supplied to a function when it is called.

Answer: arguments